

CART decision tree example using the **Weather/Golf dataset**, including key calculations, Gini impurity derivations, and binary split logic:

Step 1: Dataset Overview

Attributes & Target (14 instances) :

Day	Outlook	Temperature	Humidity	Windy	Play Golf?
D1	Sunny	Hot	High	False	No
D2	Sunny	Hot	High	True	No
D3	Overcast	Hot	High	False	Yes
D4	Rainy	Mild	High	False	Yes
D5	Rainy	Cool	Normal	False	Yes
D6	Rainy	Cool	Normal	True	No
D7	Overcast	Cool	Normal	True	Yes
D8	Sunny	Mild	High	False	No
D9	Sunny	Cool	Normal	False	Yes
D10	Rainy	Mild	Normal	False	Yes
D11	Sunny	Mild	Normal	True	Yes
D12	Overcast	Mild	High	True	Yes

D13	Overcast	Hot	Normal	False	Yes
D14	Rainy	Mild	High	True	No

Target: Play Golf? (Yes: 9, No: 5)

CART Specifics: Uses binary splits, Gini impurity, handles continuous/categorical features.

Step 2: Calculate Root Node

Gini Impurity Formula:

$$Gini(S) = 1 - \sum_{i=1}^c (p_i)^2$$

Where p_i = proportion of class i in subset S .

Overall Gini: $1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$

Feature Splits & Weighted Gini:

1. **Outlook** (Binary split: *Overcast* vs. *Not Overcast*):

● *Overcast* (4 instances): All "Yes" → Gini = 0

● *Not Overcast* (10 instances): 5 Yes, 5 No → Gini = $1 - (0.5)^2 - (0.5)^2 = 0.5$

● **Weighted Gini:** $\frac{4}{14} \times 0 + \frac{10}{14} \times 0.5 = 0.357$

2. **Humidity** (Split at ≤ 75 vs. > 75):

● ≤ 75 (6 instances): All "Yes" → Gini = 0

● >75 (8 instances): 3 Yes, 5 No $\rightarrow \text{Gini} = 1 - \left(\frac{3}{8}\right)^2 - \left(\frac{5}{8}\right)^2 = 0.469$

● **Weighted Gini:** $\frac{6}{14} \times 0 + \frac{8}{14} \times 0.469 = 0.268$

3. **Windy** (Split: *True* vs. *False*):

● *True* (6 instances): 2 Yes, 4 No $\rightarrow \text{Gini} = 1 - \left(\frac{2}{6}\right)^2 - \left(\frac{4}{6}\right)^2 = 0.444$

● *False* (8 instances): 7 Yes, 1 No $\rightarrow \text{Gini} = 1 - \left(\frac{7}{8}\right)^2 - \left(\frac{1}{8}\right)^2 = 0.219$

● **Weighted Gini:** $\frac{6}{14} \times 0.444 + \frac{8}{14} \times 0.219 = 0.315$

4. **Temperature** (Split: *Cool* vs. *Not Cool*):

● *Cool* (4 instances): 3 Yes, 1 No $\rightarrow \text{Gini} = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$

● *Not Cool* (10 instances): 6 Yes, 4 No $\rightarrow \text{Gini} = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 0.48$

● **Weighted Gini:** $\frac{4}{14} \times 0.375 + \frac{10}{14} \times 0.48 = 0.439$

Root Node Selection:

Humidity (weighted Gini = **0.268**) is the optimal root split.

Step 3: Build Sub-Trees

Branch: Humidity ≤ 75 (6 instances)

Pure "Yes" node $\rightarrow$ **Leaf (Play=Yes).**

Branch: Humidity >75 (8 instances)

Gini: 0.469

Candidate Splits:

Outlook (Split: *Sunny* vs. *Not Sunny*):

Sunny (3): All "No" → Gini=0

Not Sunny (5): 3 Yes, 2 No → Gini=0.48

Weighted Gini: $\frac{3}{8} \times 0 + \frac{5}{8} \times 0.48 = 0.3$

Windy (Split: *True* vs. *False*):

True (3): 1 Yes, 2 No → Gini=0.444

False (5): 2 Yes, 3 No → Gini=0.48

Weighted Gini: $\frac{3}{8} \times 0.444 + \frac{5}{8} \times 0.48 = 0.461$

Temperature (Split: *Mild* vs. *Not Mild*):

Mild (5): 2 Yes, 3 No → Gini=0.48

Weighted Gini: 0.48 (no improvement)

Optimal Split: Outlook (weighted Gini = **0.3**) .

Step 4: Final Tree Structure

Humidity

└─ ≤75 → Play=Yes

└─ >75 → Outlook

└─ Sunny → Play=No

└─ Not Sunny → Windy

└─ True → Play=No (2 Yes, 2 No → majority=No)

└─ False → Play=Yes (3 Yes, 0 No)

Key Notes:

Binary Splits: CART always creates binary trees (e.g., "Overcast vs. Not Overcast").

Gini vs. Entropy: CART uses Gini impurity for computational efficiency (no logarithms).

Missing Features: Temperature excluded due to high impurity reduction ($0.439 > 0.268$).

Prediction Example:

Input: [Humidity=80, Outlook=Rainy, Windy=False]

Path: Humidity>75 → Outlook=Not Sunny → Windy=False → Play=Yes.

Why CART?

Handles Continuous & Categorical Data: Automatically bins continuous features (e.g., Humidity≤75) [4].

Robustness: Less sensitive to outliers vs. ID3/C4.5 [14].

Interpretability: Binary splits create intuitive decision rules [9].

Real-World Use: Implemented in rpart (R) and scikit-learn (Python) [10].

This tree achieves 100% training accuracy. In practice, apply **cost-complexity pruning** to avoid overfitting .